SHA-512 Scheme

Course Title: Information Security Laboratory
Course No: 0714 02 CSE 4108

Submitted to,

Dr. Anupam Kumar Bairagi
Professor,
Computer Science and Engineering Discipline,
Khulna University, Khulna-9208

Submitted by,

Md. Siam Ahmed
Student ID: 230202
Year: 4th, Term: 1st
Computer Science and Engineering Discipline
Khulna University, Khulna-9208

The Implementation of SHA-512 Scheme:

```
#include <iostream>
#include <cstdint>
#include <string>
#include <sstream>
#include <iomanip>
using namespace std;

using U64 = uint64_t;

const U64 K[80] = {
    0x428a2f98d728ae22ULL, 0x7137449123ef65cdULL,
    0xb5c0fbcfec4d3b2fULL, 0xe9b5dba58189dbbcULL,
    0x3956c25bf348b538ULL, 0x59f111f1b605d019ULL,
    0x923f82a4af194f9bULL, 0xab1c5ed5da6d8118ULL,
    0xd807aa98a3030242ULL, 0x12835b0145706fbeULL,
    0x243185be4ee4b28cULL, 0x550c7dc3d5ffb4e2ULL,
    0x72be5d74f27b896fULL, 0x80deb1fe3b1696b1ULL,
    0x9bdc06a725c71235ULL, 0xc19bf174cf692694ULL,
    0xe49b69c19ef14ad2ULL, 0xefbe4786384f25e3ULL,
    0x0fc19dc68b8cd5b5ULL, 0x240ca1cc77ac9c65ULL,
    0x2de92c6f592b0275ULL, 0x4a7484aa6ea6e483ULL,
    0x5cb0a9dcbd41fbd4ULL, 0x76f988da831153b5ULL,
    0x983e5152ee66dfabULL, 0xa831c66d2db43210ULL,
    0xb00327c898fb213fULL, 0xbf597fc7beef0ee4ULL,
    0xc6e00bf33da88fc2ULL, 0xd5a79147930aa725ULL,
    0x06ca6351e003826fULL, 0x142929670a0e6e70ULL,
    0x27b70a8546d22ffcULL, 0x2e1b21385c26c926ULL,
    0x4d2c6dfc5ac42aedULL, 0x53380d139d95b3dfULL,
    0x650a73548baf63deULL, 0x766a0abb3c77b2a8ULL,
    0x81c2c92e47edaee6ULL, 0x92722c851482353bULL,
    0xa2bfe8a14cf10364ULL, 0xa81a664bbc423001ULL,
    0xc24b8b70d0f89791ULL, 0xc76c51a30654be30ULL,
    0xd192e819d6ef5218ULL, 0xd69906245565a910ULL,
    0xf40e35855771202aULL, 0x106aa07032bbd1b8ULL,
    0x19a4c116b8d2d0c8ULL, 0x1e376c085141ab53ULL,
    0x2748774cdf8eeb99ULL, 0x34b0bcb5e19b48a8ULL,
    0x391c0cb3c5c95a63ULL, 0x4ed8aa4ae3418acbULL,
    0x5b9cca4f7763e373ULL, 0x682e6ff3d6b2b8a3ULL,
    0x748f82ee5defb2fcULL, 0x78a5636f43172f60ULL,
    0x84c87814a1f0ab72ULL, 0x8cc702081a6439ecULL,
    0x90befffa23631e28ULL, 0xa4506cebde82bde9ULL,
    0xbef9a3f7b2c67915ULL, 0xc67178f2e372532bULL,
```

```
    0xca273eceea26619cULL, 0xd186b8c721c0c207ULL,
    0xeada7dd6cde0eb1eULL, 0xf57d4f7fee6ed178ULL,
    0x06f067aa72176fbaULL, 0x0a637dc5a2c898a6ULL,
    0x113f9804bef90daeULL, 0x1b710b35131c471bULL,
    0x28db77f523047d84ULL, 0x32caab7b40c72493ULL,
    0x3c9ebe0a15c9bebcULL, 0x431d67c49c100d4cULL,
    0x4cc5d4becb3e42b6ULL, 0x597f299cfc657e2aULL,
    0x5fcb6fab3ad6faecULL, 0x6c44198c4a475817ULL
};

U64 rotr(U64 x, int n)
{
    return (x >> n) | (x << (64 - n));
}

U64 sigma0(U64 x)
{
    return rotr(x, 1) ^ rotr(x, 8) ^ (x >> 7);
}

U64 sigma1(U64 x)
{
    return rotr(x, 19) ^ rotr(x, 61) ^ (x >> 6);
}

U64 bigSigma0(U64 x)
{
    return rotr(x, 28) ^ rotr(x, 34) ^ rotr(x, 39);
}

U64 bigSigma1(U64 x)
{
    return rotr(x, 14) ^ rotr(x, 18) ^ rotr(x, 41);
}

U64 Ch(U64 x, U64 y, U64 z)
{
    return (x & y) ^ (~x & z);
}

U64 Maj(U64 x, U64 y, U64 z)
{
    return (x & y) ^ (x & z) ^ (y & z);
}
```

```
void append_padding_bits_length(string &message)
{
    U64 originalBitLength = U64(message.size()) * 8;

    if (message.size() % 128 != 112)
        message.push_back(static_cast<char>(0x80));

    while (message.size() % 128 != 112)
    {
        message.push_back('\0');
    }

    for (int i = 0; i < 8; ++i)
    {
        message.push_back('\0');
    }

    for (int i = 7; i >= 0; --i)
    {
        message.push_back(
            static_cast<char>((originalBitLength >> (i * 8)) & 0xFF));
    }
}

string SHA512(string message)
{
    append_padding_bits_length(message);

    U64 a = 0x6a09e667f3bcc908ULL;
    U64 b = 0xbb67ae8584caa73bULL;
    U64 c = 0x3c6ef372fe94f82bULL;
    U64 d = 0xa54ff53a5f1d36f1ULL;
    U64 e = 0x510e527fade682d1ULL;
    U64 f = 0x9b05688c2b3e6c1fULL;
    U64 g = 0x1f83d9abfb41bd6bULL;
    U64 h = 0x5be0cd19137e2179ULL;

    for (size_t blockStart = 0;
         blockStart < message.size();
         blockStart += 128)
    {
        U64 W[80] = {};
```

```
        for (int t = 0; t < 16; ++t)
        {
            for (int j = 0; j < 8; ++j)
            {
                W[t] = (W[t] << 8) |
                       static_cast<unsigned char>(
                           message[blockStart + t * 8 + j]);
            }
        }

        for (int t = 16; t < 80; ++t)
        {
            W[t] = sigma1(W[t - 2]) + W[t - 7]
                 + sigma0(W[t - 15]) + W[t - 16];
        }

        U64 A = a, B = b, C = c, D = d;
        U64 E = e, F = f, G = g, H = h;

        for (int t = 0; t < 80; ++t)
        {
            U64 T1 = H + bigSigma1(E) + Ch(E, F, G) + K[t] + W[t];
            U64 T2 = bigSigma0(A) + Maj(A, B, C);

            H = G;
            G = F;
            F = E;
            E = D + T1;
            D = C;
            C = B;
            B = A;
            A = T1 + T2;
        }

        a += A;
        b += B;
        c += C;
        d += D;
        e += E;
        f += F;
        g += G;
        h += H;
    }
```

```
    U64 H[8] = {a, b, c, d, e, f, g, h};

    stringstream ss;
    ss << hex << setfill('0');

    for (int i = 0; i < 8; ++i)
    {
        ss << setw(16) << H[i];
    }

    return ss.str();
}

int32_t main()
{
    cout << "\nSHA-512\n";
    cout << "Enter the message: ";

    string message;
    getline(cin, message);

    string digest = SHA512(message);

    cout << "SHA-512 Scheme: " << digest << '\n';
    cout << "Scheme length: " << digest.size()
         << " hex characters\n";

    return 0;
}
```

Input/Output:

```
SHA-512
Enter the message: Implementation of SHA-512 Hashing Algorithm for
 Secure Message Digest Generation
SHA-512 Scheme: 1bbee6e6f60b2562be57b95242e4a4ab7afd6ae2707cc7faed
68d8b409db646e141fd9762fc39206f618e25fa9010c7c15bb81a21c42502c6c67
cac58e9c08fc
Scheme length: 128 hex characters
```